CIS*3110 Operating Systems

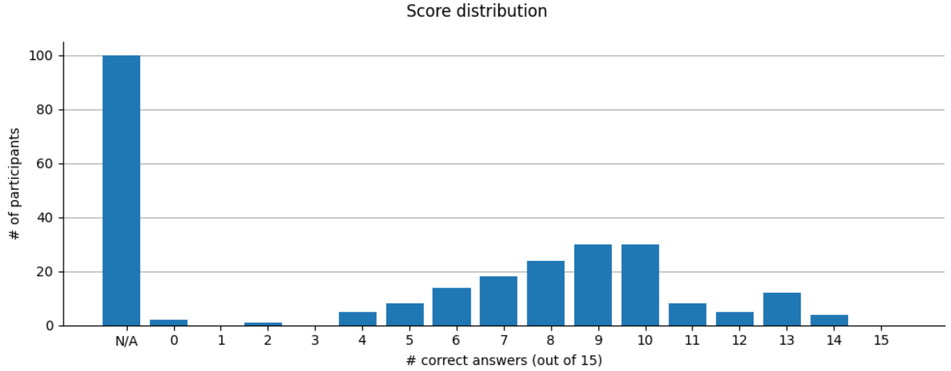
Week 5
Lecture #6 Virtualization: Memory

Prof. Yulia Kotseruba
January 27, 2025

Quiz 4

```
int main() {
    char *s = "hello";
    s[0] = 'H';
    printf("%s\n", s);
    return 0;
}
```

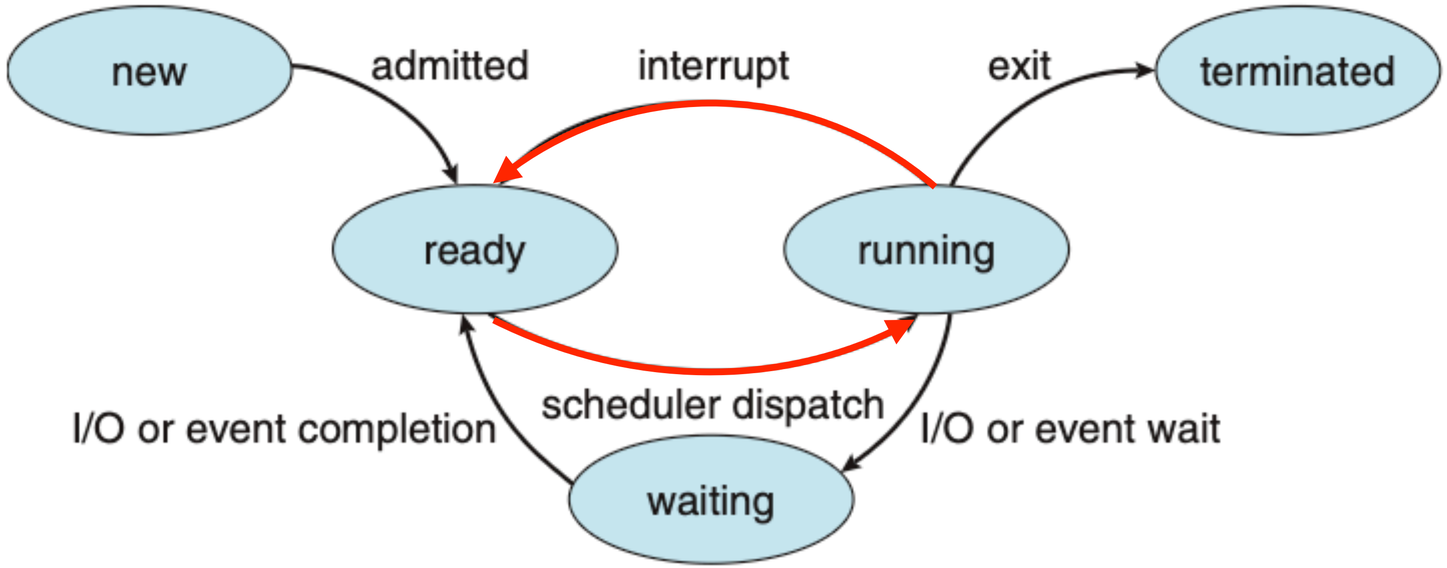
- **57%** average (down from 67% in quiz 3)
- ~~3~~ 4 hardest questions:
  - (22%) What is the output of this code? a) Hello b) hello c) compiler error d) runtime error
  - (38%) Which scheduler was used in the early batch processing OSs? a) SJF b) PSJF c) RR d) CFS
  - (38%) Starvation in MLFQ ___ a) means some jobs never run b) is when queues are empty c) never happens d) only happens in high-priority queues
  - (43%) In CFS, process with a lower nice value gets __ a) less CPU time b) more CPU time d) more vruntime d) higher priority queue



:(

Recap: Scheduling

- Scheduler has two responsibilities
  - Interrupt the current job and choose the next job to run on a CPU
  - Dispatch the next job (perform context switch if needed)
- Scheduler operates between ready and running states



Recap: Scheduling

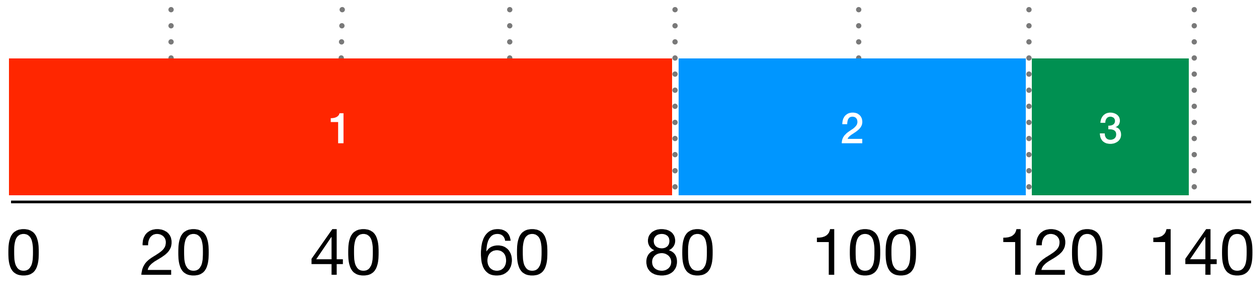
- **Non-preemptive scheduling**
  - Jobs run out until they are finished or release the CPU themselves (e.g. call sleep())
  - Does not need a PCB to monitor state
- **Preemptive scheduling**
  - Jobs can be interrupted at any time
  - PCB is needed - tracking status of the processes, context switching
  - Timer interrupts are needed to make scheduling decisions

Recap: Scheduling

- **Non-preemptive**
  - FIFO, SFJ - simple, impractical
- **Preemptive**
  - PSJF, RR - simple, low overhead, inflexible
  - MLFQ - the most general, many parameters, hard to tune
  - Lottery, Stride - simple, low overhead, but hard to set parameters (tickets/stride)
  - FCS, EEVDF - complex, low overhead, fair-share, no heuristics
    - main innovation: variable CPU share depending on the priority

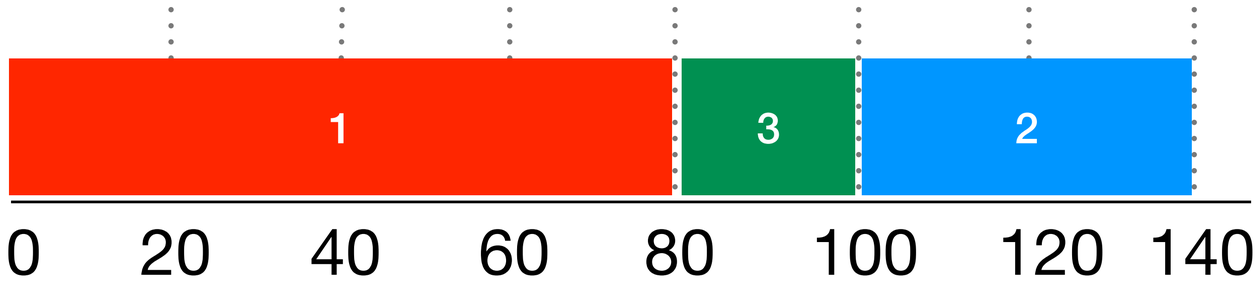
FIFO

Process	Arrival	Duration	Turnaround	Response
1	0	80	80	0
2	20	40	100	60
3	30	20	110	90



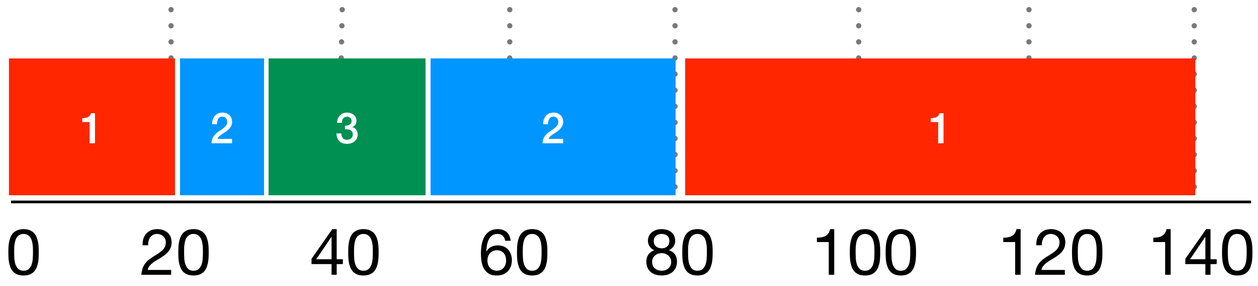
SJF

Process	Arrival	Duration	Turnaround	Response
1	0	80	80	0
2	20	40	120	80
3	30	20	70	50



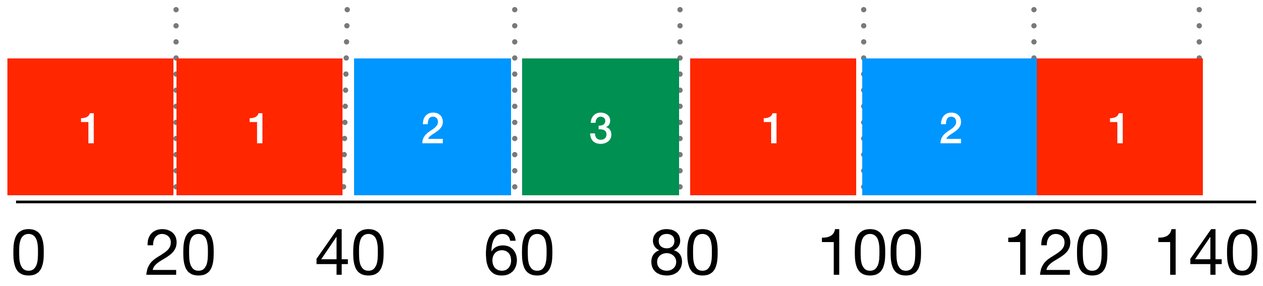
PSJF

Process	Arrival	Duration	Turnaround	Response
1	0	80	140	0
2	20	40	60	0
3	30	20	20	0



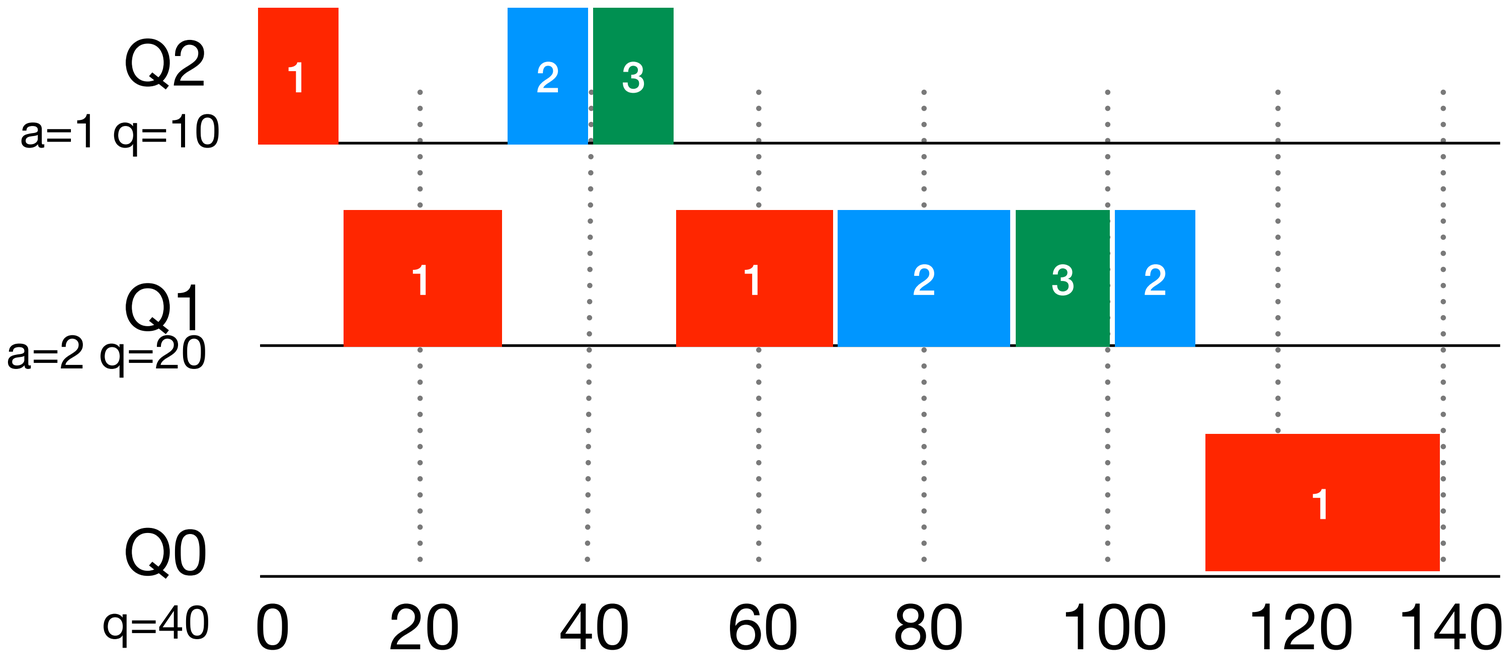
RR(q=20)

Process	Arrival	Duration	Turnaround	Response
1	0	80	140	0
2	25	40	95	15
3	30	20	50	30



MLFQ

Process	Arrival	Duration	Turnaround	Response
1	0	80	140	0
2	20	40	90	5
3	25	20	75	15

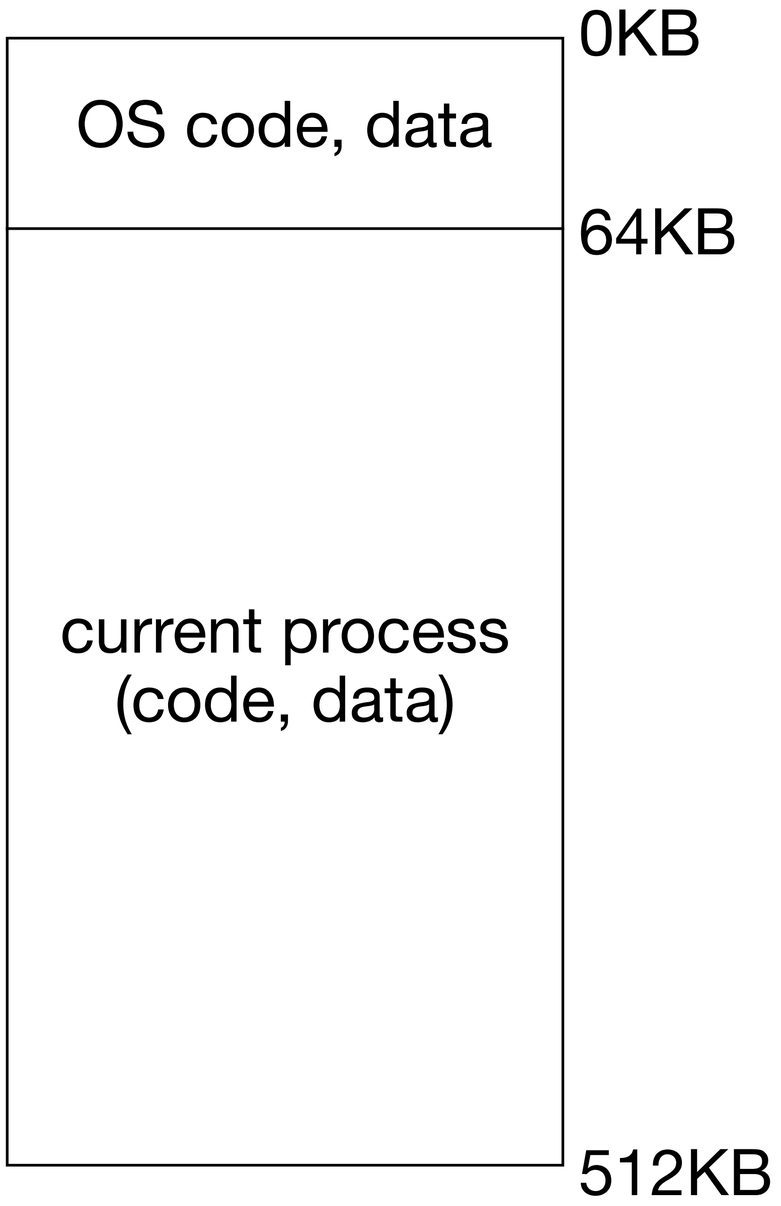


Memory

- Every process has dedicated memory for
  - instructions to be executed
  - intermediate steps of the calculation

Memory: Early days

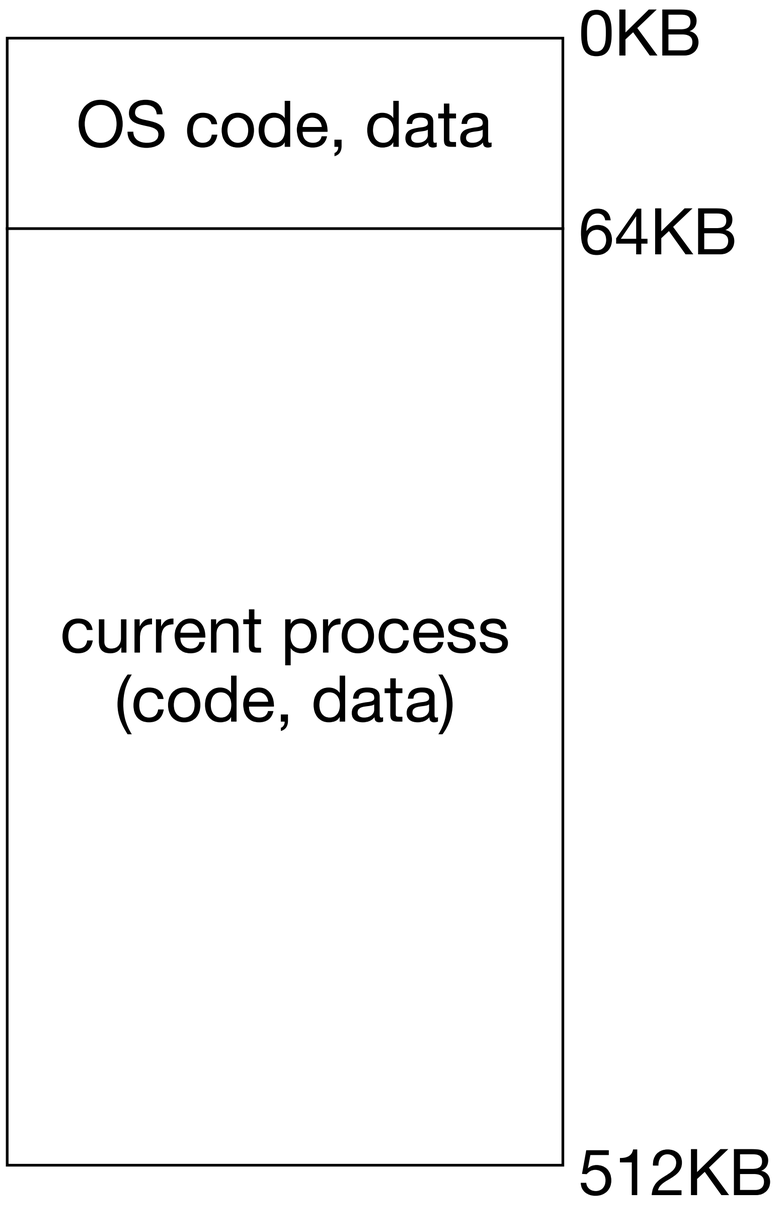
- In the early days OSs were just libraries
  - There was one (1) process in physical memory at any time
  - Once the process finished it was removed
  - The next program was loaded and ran
- Processes ran **directly on hardware**
- **No CPU or memory virtualization**



Memory: Time sharing

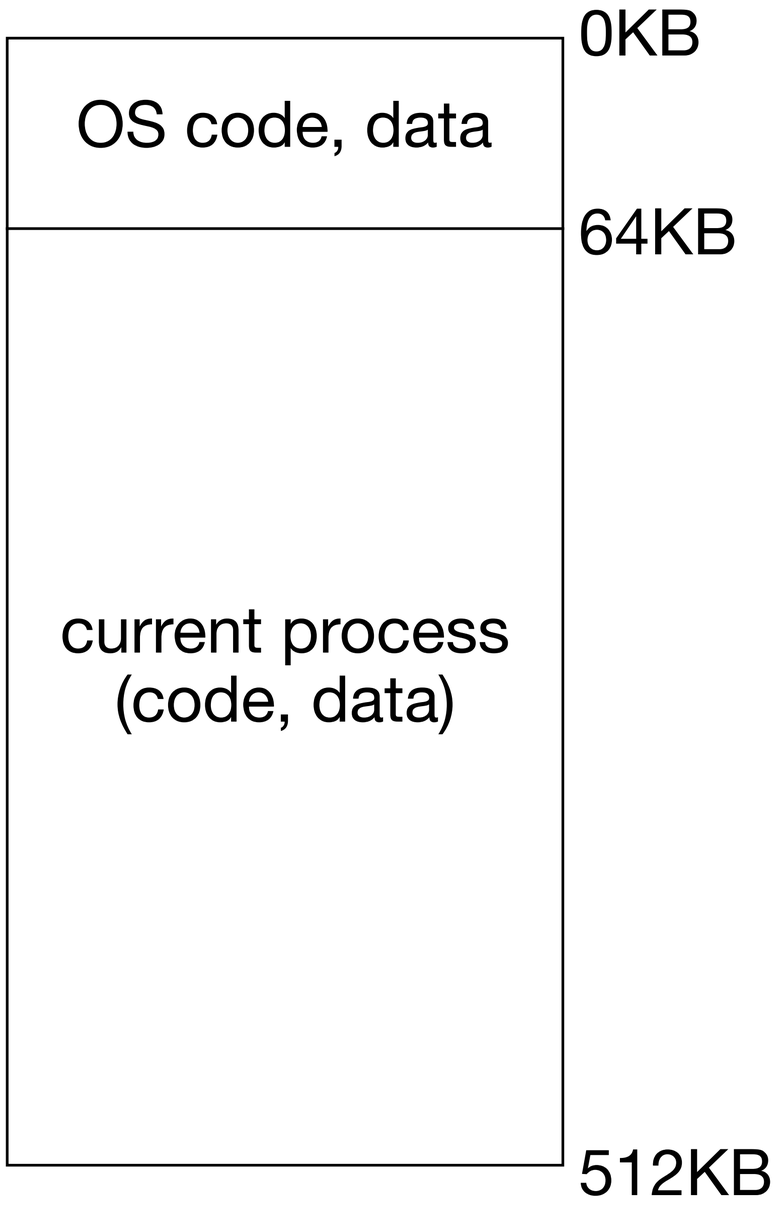
- Many users share (very expensive) machines and want results sooner than later
- Solution: **time sharing** and **context switching**

  1. Run one process for a while
  2. Save its memory
  3. Load another process
  4. Go to 1



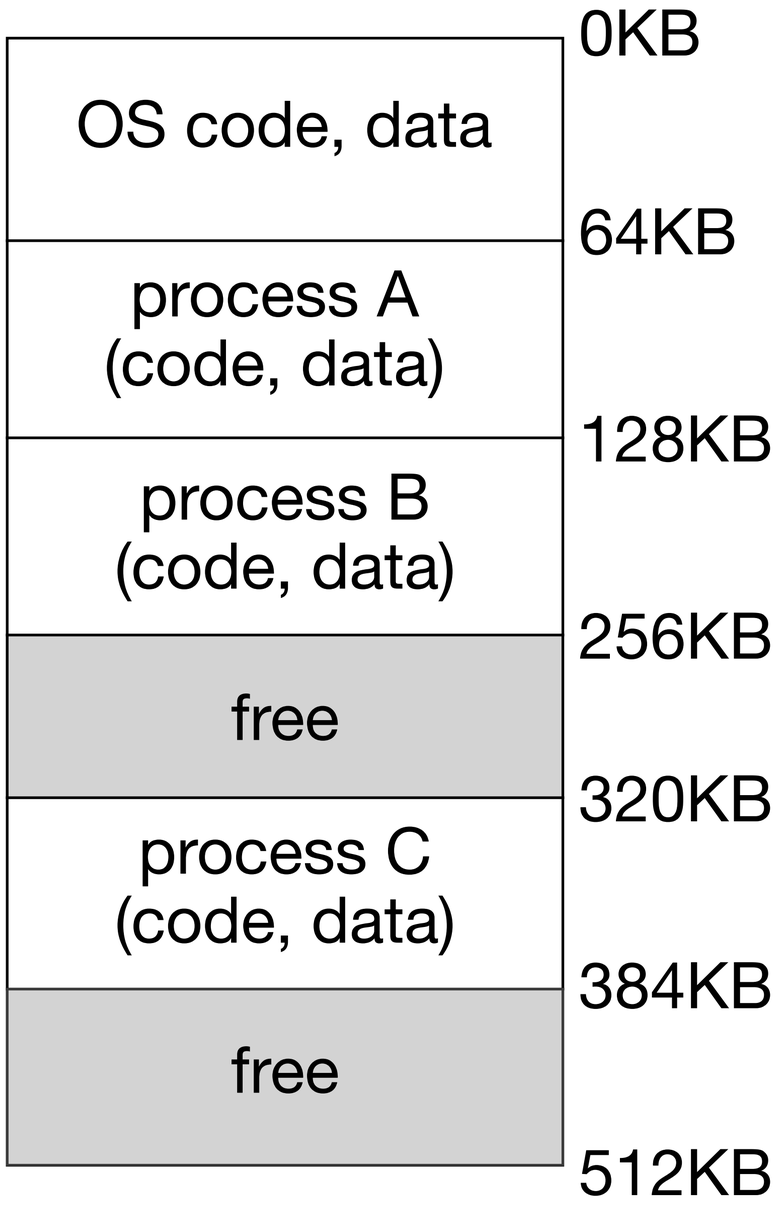
Memory: Time sharing

- Many users share (expensive) machines and want results sooner than later
- Solution: time sharing and context switching
- Problem
  - saving the whole process memory is **very slow**
- Solution: **keep processes in memory** during switch



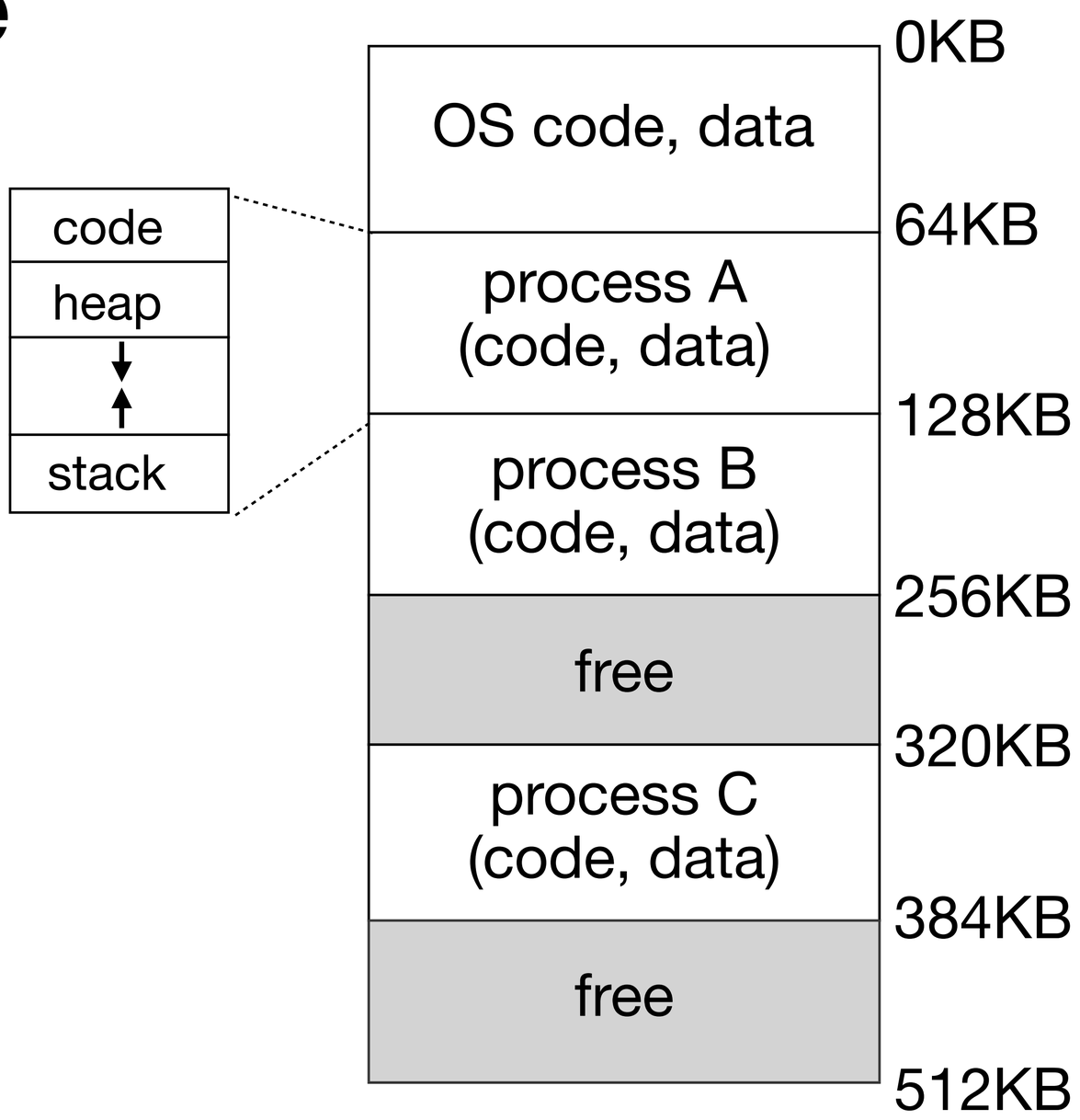
Memory: Time sharing

- Many users share (expensive) machines and want results sooner than later
- Solution: time sharing and context switching
- Problem
  - saving the whole process memory is **very slow**
- Solution: **keep processes in memory** during switch



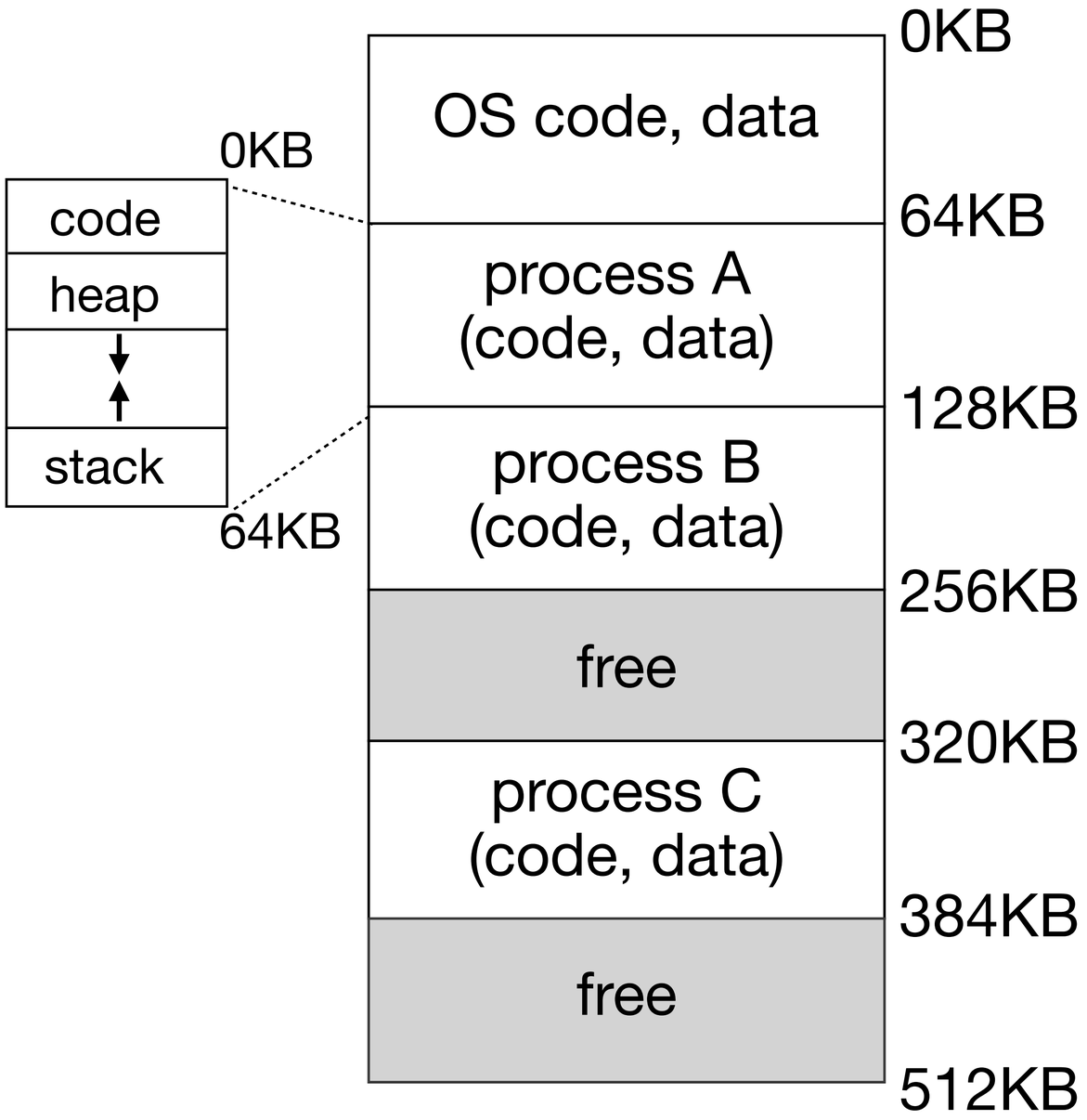
Memory: Address space

- Each process has address space
  - **code** - instructions
  - **stack** - local vars, function calls, function params
  - **heap** - dynamically allocated memory
- Code is **immutable** - placed at the beginning of address space
- Stack and heap grow in **opposite** directions



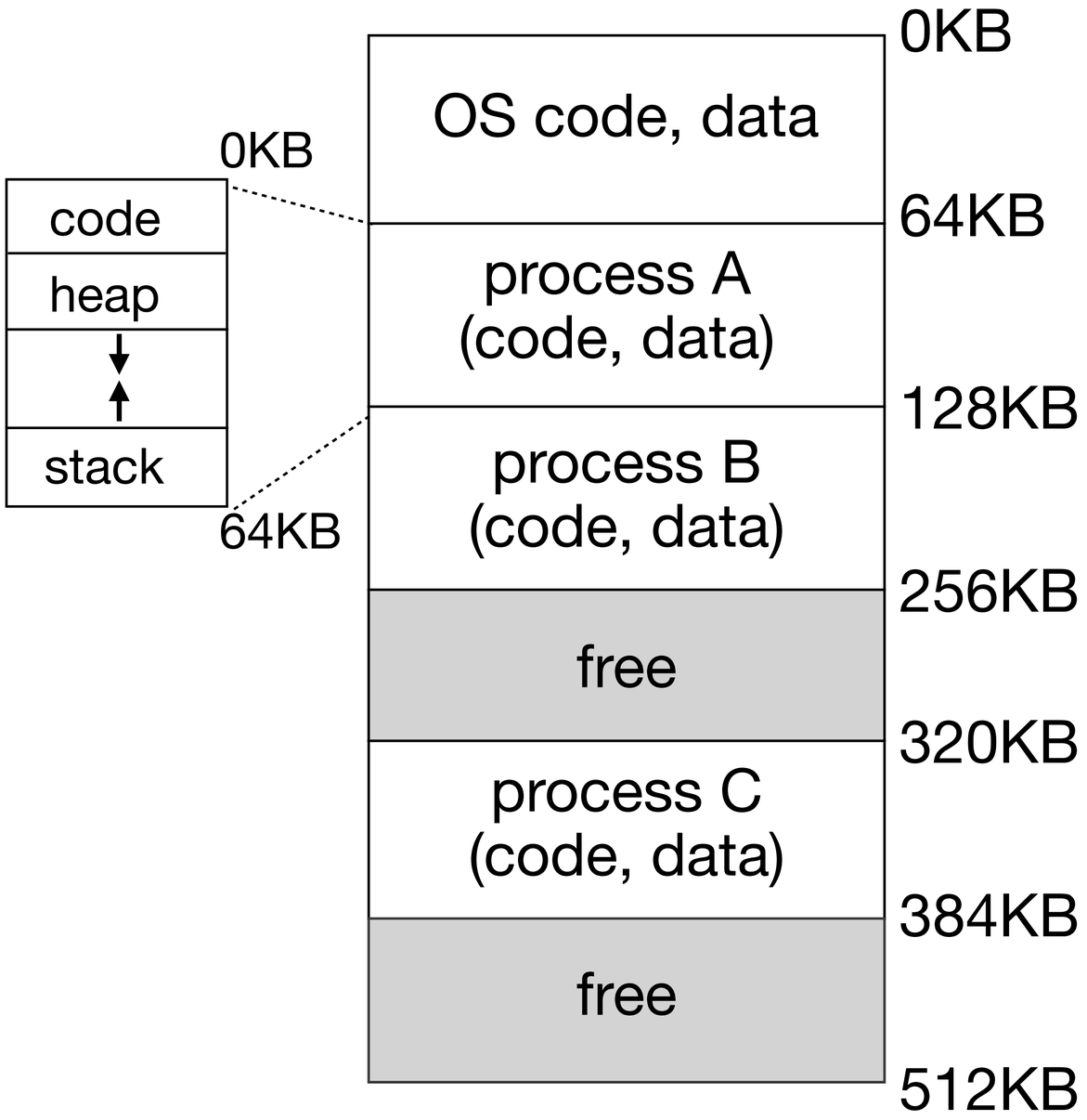
Memory: Virtualization

- **Problem #1**: addressing at **compile time**
  - compilers need addresses of heap, stack, etc. to produce instructions
  - but until the program is run, the exact location in physical memory is unknown
- **Solution #1**: give each program **virtual memory**
  - illusion of having a contiguous block of physical memory starting at 0



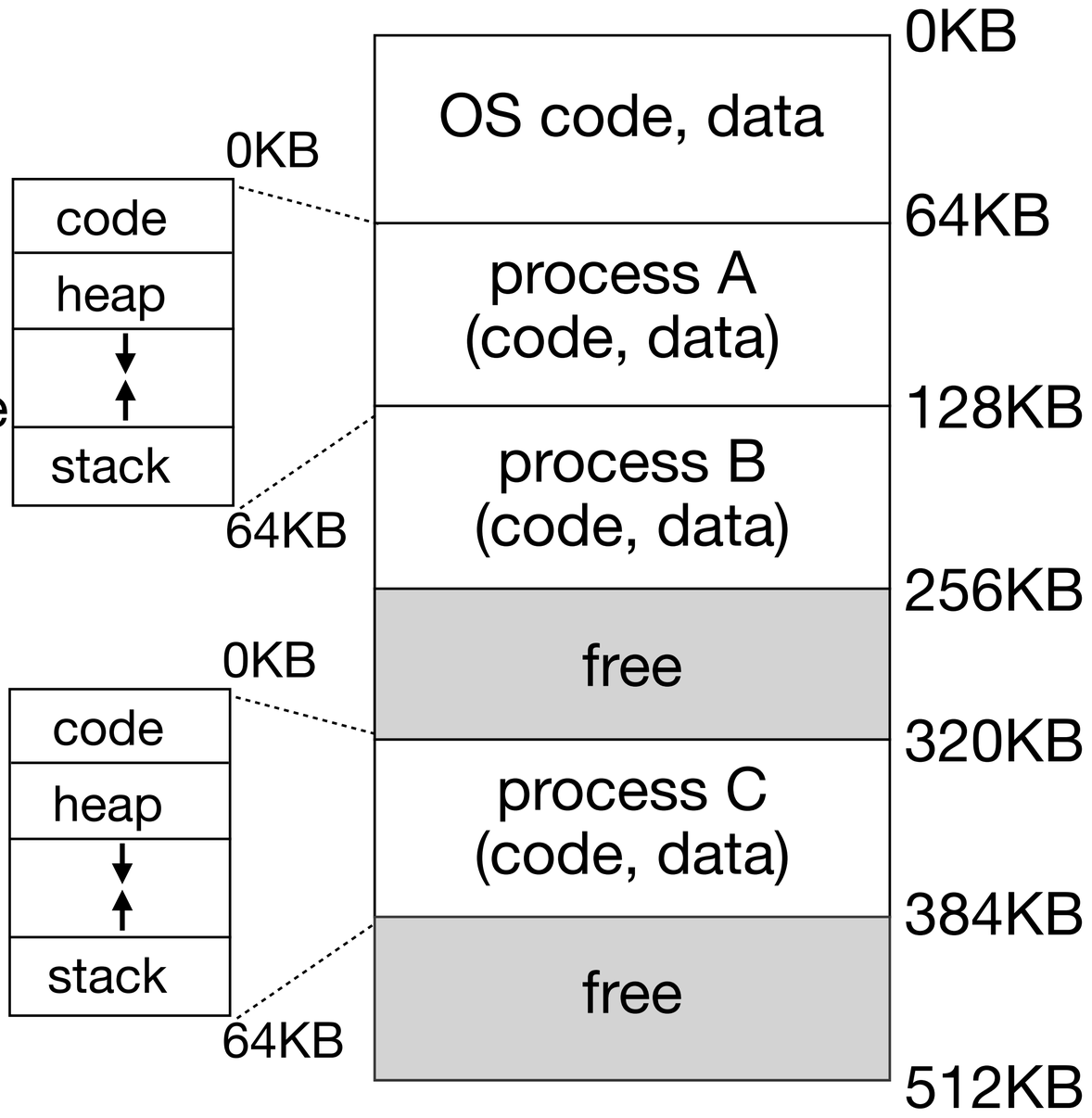
Memory: Virtualization

- **Problem #2**: addressing at **runtime**
  - How to offset address space to the right location in the physical memory?
- **Solution #2**: Let the OS handle it
  - Map the virtual addresses of the program to physical memory addresses



Memory: Virtualization

- **Problem #3**: processes do not know where they are in physical memory and how much space they have
  - How to make sure processes do not write into each other's memory
- **Solution #3**: let the OS handle it
  - keep track of where each process is located and **prevent illegal memory access**
  - **allocate** more **memory** if needed to maintain the illusion of infinite memory



Memory: Virtualization

- **Transparency*** - programs have virtual memory
  - illusion of contiguous and (almost) unlimited physical memory
- **Protection** - programs are isolated from one another
  - trespassing on the memory of other processes or the OS itself is prevented by the OS
- **Efficiency** - memory management should be time and space efficient virtualization
  - the next two weeks of lectures will cover this

*as in hard to notice, not out in the open

from Lecture 2 slides

Virtualization: memory

```
#include <stdio.h>                                  mem.c
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[]) {
    int *p = malloc(sizeof(int));
    assert(p != NULL);
    printf("(%d) address pointed by p: %p\n",
            getpid(), p);
    p* = (int) getpid();
    while (1) {
      Spin(1);
      *p = *p + 1;
      printf("(%d) p: %d\n", getpid(), *p);
    }
    return 0;
}
```

What happens if we run two copies of this program concurrently?

PIDs are different — these are two independent processes

```
>>> ./mem & ./mem &
(2718) address pointed by p: 0x5555555592a0
(2719) address pointed by p: 0x5555555592a0
(2718) p: 2719
(2719) p: 2720
(2718) p: 2720
(2719) p: 2721
(2718) p: 2721
(2719) p: 2722
(2718) p: 2722
(2719) p: 2723
^C
```

2719 and 2720 appear to be stored at the same address

This is memory virtualization — each program has its own (virtual) address space, which is mapped by the OS to two different physical locations

For this to work, turn-off address-space layout randomization (ASLR)

Virtualization: memory

```
#include <stdio.h>                    addresses.c
#include <stdlib.h>

int main(int argc, char *argv[]) {
    printf("address of code: %p\n", main);
    printf("address of heap: %p\n", malloc(100e6));
    int x = 3;
    printf("address of stack: %p\n", &x);
    return x;
}
```

```
>>> ./addresses
address of code: 0x1023f4460
address of heap: 0x11d000000
address of stack: 0x16da0afdc
```

Virtualization: memory

```
#include <stdio.h>                          addresses.c
#include <stdlib.h>

int main(int argc, char *argv[]) {
    printf("address of code: %p\n", main);
    printf("address of heap: %p\n", malloc(100e6));
    int x = 3;
    printf("address of stack: %p\n", &x);
    return x;
}
```

User-level processes do not know their physical addresses

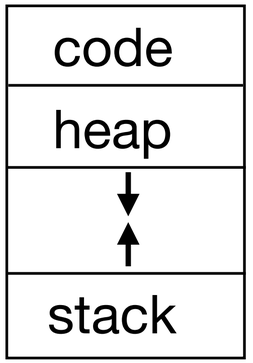
Any address printed from a program will be virtual, only the OS knows where it is in physical memory

```
>>> ./addresses
address of code: 0x1023f4460
address of heap: 0x11d000000
address of stack: 0x16da0afdc
```

HEX	DEC
0x1023f4460	4332668000
0x11d000000	4781506560
0x16da0afdc	6134214620

Relative locations are preserved

Code is first, then heap and then stack.



There are better tools for examining memory in C, which will be covered in Lab 4

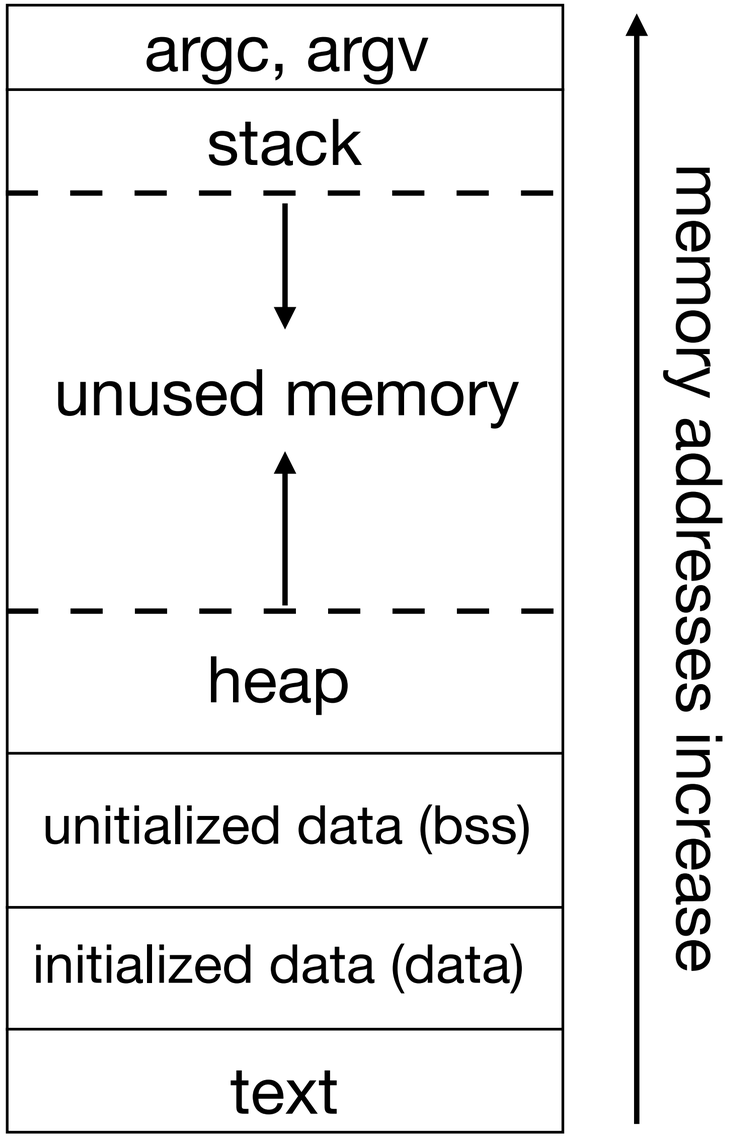
Summary so far

- Most modern OSs implement **virtual memory**
  - creates an **illusion** of **large**, **contiguous**, **private** space for each process
  - the OS **knows** where the processes are located in **physical memory**
  - the OS **manages** memory of the processes to prevent **illegal accesses**
- Memory management is done through **hardware** and **software** mechanisms

from Lecture 3 slides

Process memory

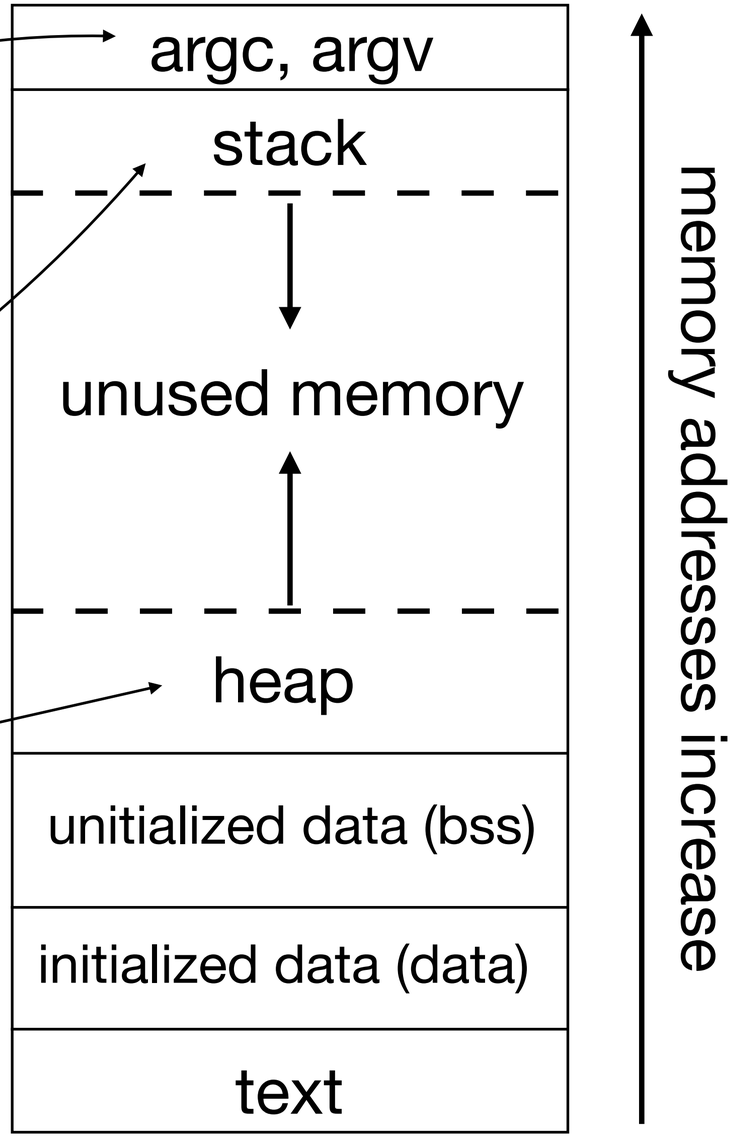
- **Text** segment - executable code, read-only, fixed-size
- **Data** segment - global/static variables
  - Initialized (data) and unitilialized (bss) data
- **Heap** segment - dynamically allocated memory, maintained by malloc and free, modifiable at runtime
- **Stack** segment - temp data storage, fixed-size
  - stack pointer (SP) - points to the top of the stack
  - automatic variables - function params, return addresses
  - local variables - e.g. statically sized arrays



Process memory

```c
#include <stdio.h>
#include <stdlib.h>
int x;
int y = 15;

int main(int argc, char *argv[]){
  int *values;
  int i;

  values = (int *) malloc(sizeof(int)*5);
  for (i = 0; i < 5; i++)
    values[i] = [i];
  return 0;
}
```

mem.c



p.108 from Silberchatz et al 10ed.

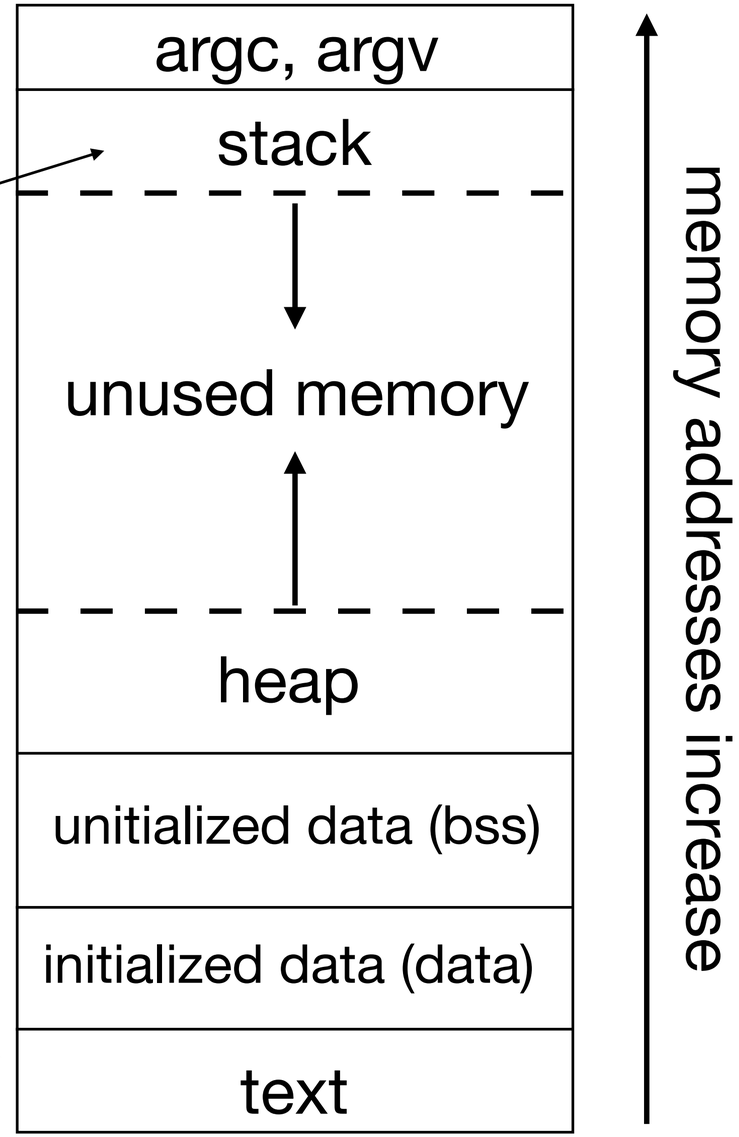
Process memory

```
#include <stdio.h>
#include <stdlib.h>
int x;
int y = 15;

int main(int argc, char *argv[]){
  char s1[] = "Hello";
  char *s2 = "Hello";

  int *values;
  int i;

  values = (int *) malloc(sizeof(int)*5);
  for (i = 0; i < 5; i++)
    values[i] = [i];
  return 0;
}
```

mem.c

string is used to initialize array of chars

s2 pointer is on the stack but points to read-only data



Memory API

- Any process has a **stack** and a **heap**
- Stack is managed by the **compiler** (automatic memory)
  - variable declarations (e.g. int x;) are allocated/deallocated automagically by the compiler without programmer's involvement
- Heap is managed by **you** (the programmer)
  - `malloc()`, `calloc()`, `realloc()`, `free()`

`malloc()`

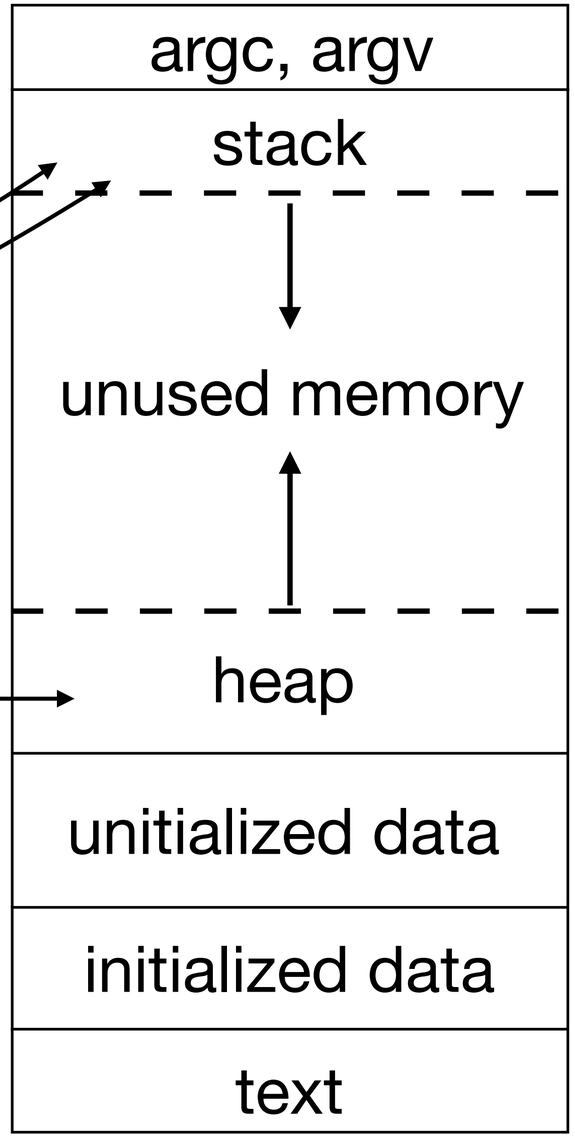
- `malloc()` - **m**emory **alloc**ation - a function that allocates
  - a **single contiguous unitialized** block of memory
  - on the heap
  - at runtime
- Allocated memory **persists** even after the function finishes
  - unlike anything allocated on the stack, which is cleaned up after return
- `calloc()` - allocates and zero-initializes the memory
- `realloc()` - extends allocated memory (allocates a new larger region and copies the contents of the old one there)

malloc()

Where are x and y located in process memory?
y - stack, x - stack, memory returned by malloc() - heap

malloc_test.c

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){
  int y[10];


  int *x = malloc(10 * sizeof(int));


  return 0;
}
```



malloc()

malloc_test.c

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){
  int y[10];
  printf("%d\n", sizeof(y));

  int *x = malloc(10 * sizeof(int));
  printf("%d\n", sizeof(x));

  return 0;
}
```

What is the output?

```
>>> ./malloc_test
40
4
```

why?

compiler knows size of y at compile time and converts it to a constant (10*sizeof(int))

x is allocated at runtime, all sizeof knows is the size of the pointer itself

I made a mistake in class, sizeof will return the amount of memory for the array not the number of elements

malloc()

malloc_test.c

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){
  int y[10];
  printf("%d\n", sizeof(y));

  int *x = malloc(10 * sizeof(int));
  printf("%d\n", sizeof(x));

  return 0;
}
```

Is anything missing in this code?

free() should be somewhere here

`free()`

- free() **deallocates** memory
  - expects a **pointer** returned by **malloc()**
  - **does not** take the size of the allocated region
    - size is stored by the malloc() adjacent to the allocated block (lab 4)
  - marks the block as available for **reuse**
- C does not have automatic memory management (aka garbage collection)
- Each program must allocate and deallocate its heap memory
- Many mistakes can arise

Allocating memory

ok_malloc.c

```
typedef structure Data {
  int one;
  int two;
} Data;

Data* from_stack() {
  Data data = {1,2};
  Data *heap_data = malloc(sizeof(Data));
  memcpy(heap_data, &data, sizeof(Data));
  return heap_data;
}

int main(int argc, char *argv[]){
  Data *heap = from_stack();
  printf("one: %d, two: %d\n", heap->one, heap->two);
}
```

```
>>> ./ok_malloc
one: 1, two: 2
```

there is nothing wrong with this code

from_stack() makes a data struct and initializes it on the stack

then allocates memory on the heap, copies the struct there, and returns the pointer to main

print prints the contents of data

Not allocating memory

no_malloc.c

```
typedef structure Data {
  int one;
  int two;
} Data;

Data* from_stack() {
  Data data = {1,2};
  Data *heap_data = memcpy(heap_data, &data,
  sizeof(Data));
  return heap_data;
}

int main(int argc, char *argv[]){
  Data *heap = from_stack();
  printf("one: %d, two: %d\n", heap->one, heap->two);
}
```

What if we forgot to call malloc() before memcpy?

```
>>> ./no_malloc
Bus error: 10
```

Bus error means accessing invalid address, which causes a hardware interrupt

Bus error is not the same as segfault - invalid operation on valid address

the type of error may be C-implementation specific, but either way this code is doing something very bad

Not allocating enough memory

not_enough.c

```
int main(int argc, char *argv[]){
  char *src = "hello";
  char *dst = (char *) malloc(strlen(src));
  strcpy(dst, src);
  printf("src: %src  dst: %dst\n", src, dst);
  return 0;
}
```

a common mistake, need 1 more char for '\0'

```
>>> ./not_enough
hello hello
```

Not allocating enough memory (aka buffer overflow) won't trigger compiler errors and may run normally

This may however corrupt heap and lead to hard-to-trace bugs

e.g. malloc() stores additional block info in the adjacent area, so free() may stop working if that info is corrupted

Not initializing memory

```
typedef structure Data {                    no_init.c
  int one;
  int two;
} Data;

Data* from_stack() {
  Data data = {1,2};
  Data *heap_data = malloc(sizeof(Data));
  return heap_data;
}

int main(int argc, char *argv[]){
  Data *heap = from_stack();
  printf("one: %d, two: %d\n", heap->one, heap->two);
}
```

What if we do not put anything in the heap_data?

```
>>> ./no_init
one: 0, two: 0
```

Will compile and run

What is printed depends on what was stored in that area before

Unitialized memory can contain anything

Anything wrong here?

```
//includes here

struct Stuff {
  int number;
} Stuff;

void do_something_nasty() {
  Stuff* ugly = malloc(sizeof(Stuff));
  ugly->number = 10;
}

int main(){
  do_something_nasty();
}
```

allocate memory pointed by the pointer on the stack

that pointer will be gone from the stack after do_something_nasty() returns

main would not be able to free it even if it wanted to

memory leak!

Not freeing memory

What if we forgot to call free()?

nasty.c

```
//includes here

struct Stuff {
  int number;
} Stuff;

void do_something_nasty() {
  Stuff* ugly =
  malloc(sizeof(Stuff));
  ugly->number = 10;
}

int main(){
  do_something_nasty();
}
```

```
>>> ./nasty
>>> valgrind --leak-check=full \
--show-leak-kinds=all \
--track-origins=yes ./nasty
==305186== HEAP SUMMARY:
==305186==     in use at exit: 4 bytes in 1 blocks
==305186==   total heap usage: 1 allocs, 0 frees, 4
bytes allocated
==305186==
==305186== 4 bytes in 1 blocks are definitely lost in
loss record 1 of 1
==305186==    at 0x4846828: malloc
==305186==    by 0x10915E: do_something_nasty
==305186==    by 0x109181: main
==305186==
==305186== LEAK SUMMARY:
==305186==    definitely lost: 4 bytes in 1 blocks
==305186==    indirectly lost: 0 bytes in 0 blocks
==305186==      possibly lost: 0 bytes in 0 blocks
==305186==    still reachable: 0 bytes in 0 blocks
==305186==         suppressed: 0 bytes in 0 blocks
```

ugly pointer is gone after do_something_nasty returns. After that the memory cannot be freed anymore

```
//includes here
void *p;
int main(){
  p = malloc(10);
  p = malloc(100);
  void *m = malloc(50);
  return 0;
}
```

What is happening here?

second malloc call overrides the address pointed at by p with the new address

the originally allocated memory cannot be freed now because that pointer is lost

in this particular case it won't do any harm because OS will clean up the memory after the process finishes

but if a large chunk of memory is lost repeatedly, a long-running program might run out of memory completely

Not freeing memory

What is happening here?

lost.c

```
//includes here
void *p;
int main(){
  p = malloc(10);
  p = malloc(100);
  void *m = malloc(50);
  return 0;
}
```

```
>>> ./lost
>>> valgrind --leak-check=full \
--show-leak-kinds=all \
--track-origins=yes ./lost
==305585== LEAK SUMMARY:
==305585==    definitely lost: 60 bytes in 2 blocks
==305585==    indirectly lost: 0 bytes in 0 blocks
==305585==      possibly lost: 0 bytes in 0 blocks
==305585==    still reachable: 100 bytes in 1 blocks
==305585==         suppressed: 0 bytes in 0 blocks
```

10 bytes are lost because of the second call to malloc()

50 bytes are lost because m* is in the local scope (on the stack) and does not exist after return

100 bytes are still referenced by the global pointer p - still reachable

Not freeing memory

What if we forgot to call free()?

lost.c

```
//includes here
void *p;
int main(){
  p = malloc(10);
  p = malloc(100);
  void *m = malloc(50);
  exit(0);
}
```

```
>>> ./lost
>>> valgrind --leak-check=full \
--show-leak-kinds=all \
--track-origins=yes ./lost
==305441== LEAK SUMMARY:
==305441==    definitely lost: 10 bytes in 2 blocks
==305441==    indirectly lost: 0 bytes in 0 blocks
==305441==      possibly lost: 0 bytes in 0 blocks
==305441==    still reachable: 150 bytes in 2 blocks
==305441==         suppressed: 0 bytes in 0 blocks
```

Why is there a difference?

return exits the current function - pointers in the local scope are not accessible - definitely lost leaks

exit() terminates the program and does cleanup immediately - still has access to the pointers in the local scope - those are still reachable

Not freeing memory

- OS will reclaim all memory and resources on exit
  - many implementations of `free()` do not return memory to the OS anyway
- It is good practice to explicitly free all memory
  - will help catch real memory leaks detected by memory tracing programs (e.g. valgrind)
  - if you anticipate your program to be running for a long time

`malloc() + free()`

- Do not forget to allocate enough memory - avoid overwriting existing memory and buffer overflow
- Do not forget to initialize memory - a random chunk of allocated memory can contain anything in it
- Do not forget to free memory - avoid memory leaks*
- Do not free the same memory repeatedly - process will be aborted
- Do not free memory not allocated with malloc() - process will be aborted
- Do not forget to check malloc() return value

*but also understand when it is better to leave clean-up to the OS

Summary

- Processes see their memory as a **contiguous** and **virtually unlimited**
- Processes are **not aware** of their memory location in **physical memory**
- **OS manages allocation** and **access** to memory to **protect** processes from each other and itself from processes

Summary

- There are 3 types of memory allocation for processes
  - Each process gets memory allocated by the OS
  - Within the allocated space, the process can allocate/deallocate memory in the heap section
  - Stack memory is managed by the compiler
- This is C-specific, other languages may have garbage collection to manage their heap allocations as well

Reading

- Chapter 13 The Abstraction: Address Spaces
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-intro.pdf
- Chapter 14 Interlude: Memory API
  - https://pages.cs.wisc.edu/~remzi/OSTEP/vm-api.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/vm-intro
- Optional reading
  - https://c-faq.com/index.html
  - https://danluu.com/malloc-tutorial/

A1

- You can develop on your own machine and any OS
- autograder will test your code on SoCS servers automatically and push the report into A1feedback branch in your repo
- Do not commit binary files (*o files and executables) into your repos
- If you are using printf to debug, make your life easier with macros
- Test #71 will be rerun after the submission deadline
  - There are several large courses using autograders this term, we are competing for resources with them and things get slow time to time
  - If your code passes all 70 tests but fails #71 due to timeout, it will be rerun on a local server to accurately measure execution time

Announcements

- **SOCIS URA/USRA Mixer**
  - this Thursday, Jan 29, 1-3pm @ REYN1101
  - No office hours on Jan 29
- Notetakers needed
  - Please sign up if you are taking notes already
- Communication
  - If you ask me for something, please give me enough information to act on
  - If your issue was resolved, please let me know
  - Please post non-personal questions on Teams

Quiz 5

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999